Managing Docker networks with commands

In this train, we'll become proficient in using Docker network commands to inspect, create, modify, and delete networks.

Learning objectives

- Use Docker network commands to manage networks effectively.

How to manage a Docker network effectively?

Docker networks allow containers to communicate efficiently while maintaining isolation where needed. Managing these networks effectively requires a strong understanding of [Docker's network commands.](https://docs.docker.com/reference/cli/docker/network/).

> **Prerequisites:**
> - Have a network created.
> - To make sure that your network has been created: `docker network ls`

1. Inspecting a Docker Network

- To check details about a specific network, use:

  `docker network inspect my-custom-network`

  > This command provides information about the network configuration, connected containers, and IP range.

2. Connecting a Container to a Network

- To attach a running container to a specific network:

  ```
  docker network connect my-custom-network my-container
  ```

  > **Explanation:**
  > - `my-custom-network` : The target network.
  > - `my-container` : The container to connect.

3. Disconnecting a Container from a Network

- To detach a container from a network:

  ```
  docker network disconnect my-custom-network my-container
  ```

4. Removing a Docker Network

- To delete a network, first ensure no active containers are connected, then run:

  ```
  docker network rm my-custom-network
  ```

  > **Note:** You cannot remove a network if containers are still attached.

Conclusion

Mastering Docker network commands is essential for effectively managing container communication. By creating, inspecting, modifying, and deleting networks, you can ensure your applications run smoothly within controlled networking environments.

Key Takeaways

- Inspect network configurations with `docker network inspect`.
- Manage container connections with `docker network connect` and `docker network disconnect`.

- Remove unused networks using `docker network rm`.